

Sadi Sir

Lecture-1

TOPIC NAME: Introduction

DAY: Sunday

TIME:

DATE: 3 / 8 / 25

Ref Books:

① Tcp/IP Protocol Suite (4th Edition)

- Forouzan

② Computer Networking Top Down Approach (8th)

- Ross, Kurose

Lecture-2

Sunday

10 / 8 / 25

Chapter-2

↳ [Forouzan]

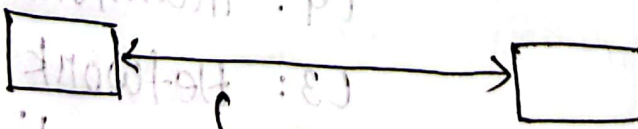
OSI Model & TCP/IP suite

⇒ OSI:

→ OSI model ଏକା Protocol. ଯିଏ କିଏ device ଏ
Protocol ଭାବେ କରେ, ତାହା n/w ଏବଂ ସାଥେ connect
ହେବା ପାଇଁ ଚାହୁଁଥାଏ ।

⇒ TCP/IP Protocol suite:

↳ Another set of Protocol



କିମ୍ବା commⁿ ହେବା ପାଇଁ OSI / TCP/IP
Protocol.

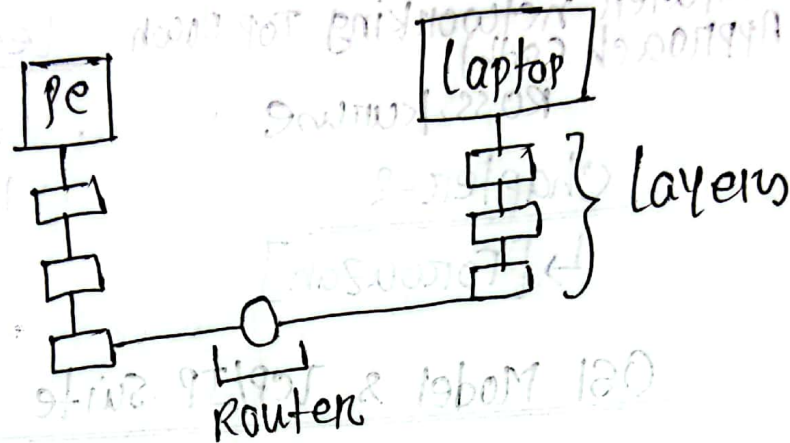
TOPIC NAME : _____

DAY: _____

DATE: / /

TIME: _____

→ ଦୁଇଟି device ବା commⁿ ବା ଜନ) ଶାଖେ ଏକତର layer
 ଶାଖା ।



* OSI Model ଗ୍ରନ୍ଥ middlewear.

OSI - Model layers: Ter/IP Model layers:

7 layers -

5 layers

L7 : Application

L6 : Presentation

L5 : session

L4 : Transport

L3 : Network

L2 : Data link

L1 : Physical

L5 : Application

Presentation
 session

L4 : Transport

L3 : Network

L2 : Data link

L1 : physical.

} Application

TOPIC NAME : _____

DAY: _____

TIME: _____

DATE: / /

- * Send ও layer top to Bottom.
- * Receive " " Bottom to top.
- * প্রত্যেক layer at most দুই adjacent layer ও অন্য commⁿ করে পারে।
- * প্রত্যেক layer data ও অন্য extra header + tailen add করে।

(***) OSI vs TCP/IP

	OSI	TCP/IP
Innovation	New	Old
Used	less	Much

⇒ Main ও TCP/IP model

Key Terms:

① IP Address → H/W layer (L3)

② Physical Address (MAC) → Data link layer (L2)

③ Port number / Address → Transport layer (L4)

kiire?	S	D
--------	---	---

TOPIC NAME: _____

layerData ७३ तल

* Application → message

* Transport → segment

* Network → datagram

* Data link → frame

* physical → bits.

Chp-2

Lecture-3

TOPIC NAME: Application Layer

DAY: Wednesday

TIME:

DATE: 13 / 8 / 25

→ Rules & Regulation নিয়ম আলাচনা করা হয়।

Principle
of H/w App

web & HTTP

→ User তর internet we করতে হয়।

Application layer
S ↓ R ↑
Transport layer

✳ Email → SMTP

web → HTTP

File → FTP (ফাইল স্থানান্তর)

✳ Creating a network app:

① write program

② No need to write software for n/w each device

✳ Application Architecture:

① client server

② peer to peer

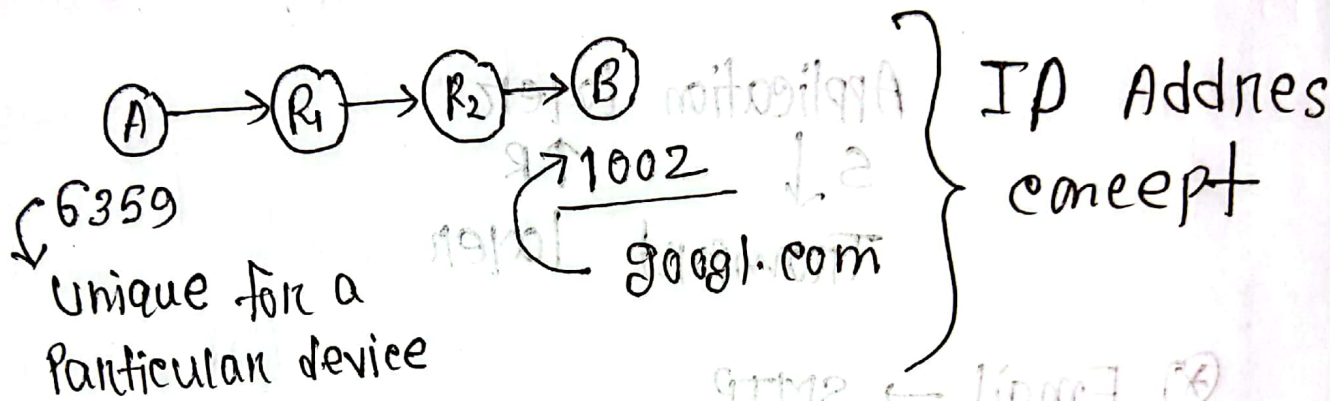
Client server

Server :

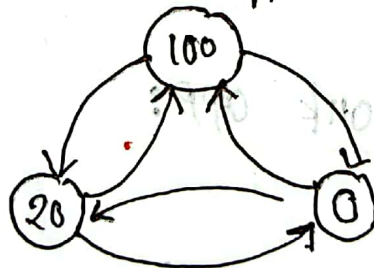
- ① Server always on
- ② Permanent IP Add

client :

- ① client = end user.



Peer to Peer



Process commⁿ

→ process running application and process.

Socket

Application takes msg from 23456 Doot.

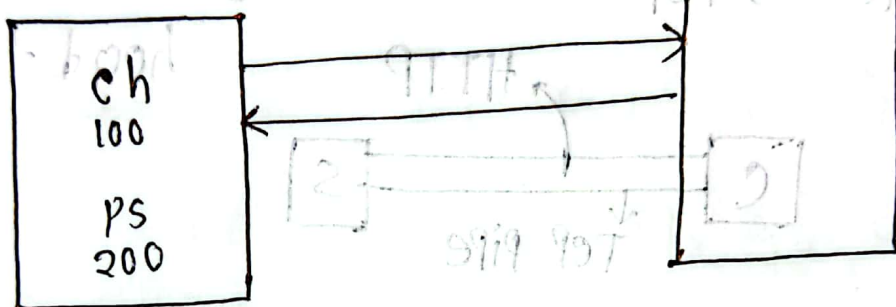
Ports

→ ~~Port~~ ports have numerical values.

Addressing values

Client (1001)

YT (2452)



⇒ IP Address : Port Number

HyperText Transfer Protocol

→ অনেকটা client server architecture এর মতো।

Protocol : // host : Port / Path

↓
Method (http, https)

⊗ HTTP → Port = 80

⊗ HTTPS → Port = 443

⊗ HTTP uses TCP

⊗ HTTP is stateless → কোরেক্ট করে, তা মনে রাখো না।

Sadi Sir

Chp-2

Lecture-4

TOPIC NAME: Application layer

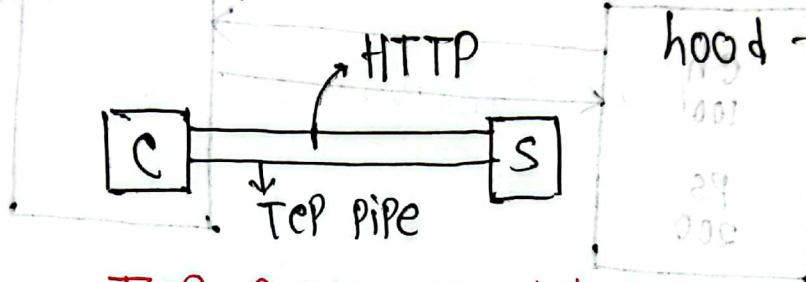
DAY: Sunday

TIME: DATE: 17/8/20

Round Trip Time (RTT)

RTT = C $\rightarrow$ S Req karta hai, Receive karta hai
↪
time.

⊗ Reliable $\rightarrow$ Tcp



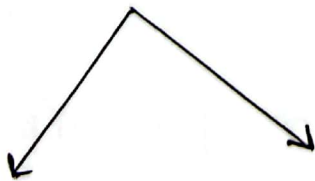
⊗ HTTP under the hood tcp me karta hai

Tcp 3-way Handshake

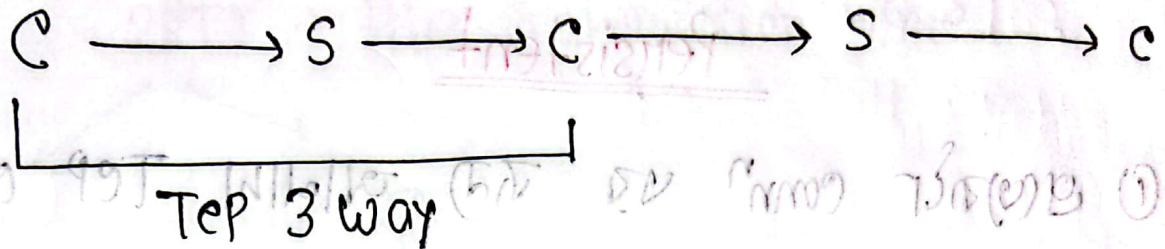
C $\xrightarrow{\text{SYN}}$ S $\rightarrow$ C $\rightarrow$ S

HTTP Connection

Non persistent



⇒ Without a parallel connection:



⊛ At a time kahi HTTP Req execution hoto.

eg: 1 web page } 1st bar kahi web page k
20 image } load hoto.

execution hoto or execution close hoto.

⊛ Sequentially execution hoto.

⇒ A parallel connection:

⊛ Parallely kahi hoto.



⊛ Non Persistent : New connection + file transfer.

Times = (n) * 2

(***) Diagram এর explanation.

Persistent

১) প্রত্যেকের conn এর জন্য আলাদা TCP connection open করা লাগে না। একবার connection establish করলে হয়।

⊗ Persistent : 1 connection + (Transfer) * 20

⇒ Non Pipeline :

Ack পাওয়ার পরেই আরও Res পাঠানো।

⇒ Pipeline :

একবার সব send করে দিব।

(***) Advantage, Disadvantage, Difference এটা

লিখতে হয়।

⊗ Conn establish করতে RTT এর সময় লাগে।

⊗ Non persistent HTTP Resume time

$$= 2RTT + \text{File transmission time}$$

Conn
establish

File vs
Riz bit outscu

1 wp + 10 img → (Non persistent)

$$2RTT + (2RTT + \text{Tran}^m \text{ time}) * 10 = \text{Total time.}$$

Persistent

$$\text{Total time} = 2RTT + (RTT + \text{Tran}^m \text{ time}) * 10$$

HTTP Protocol - Message Format

C

S

→ 2/1/2017
2/1/2017
2/1/2017

HTTP status code

201 : Status OK

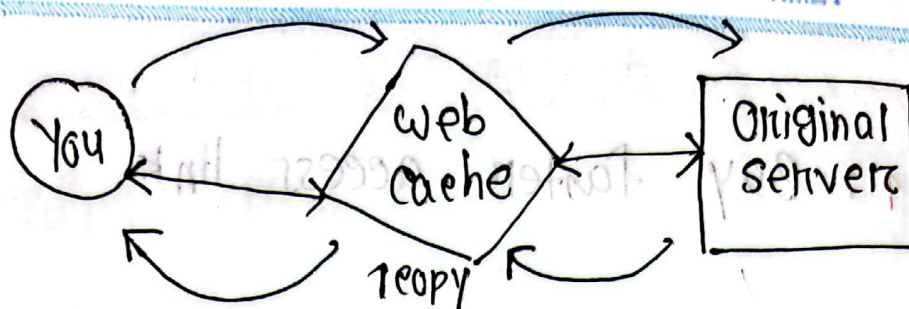
404 : Not found

cookies

→ HTTP stateless, but cookie is used to maintain the state.

→ First party cookie } (***)
→ Third party cookie }

↳ 3rd party cookie is used to track user. (***)



* Web cache এর কাজ, অর্থাৎ এর component থাকে না।

* Hit miss রকম, Original server এর কাজে Req করে, then তার একটি copy নিজের কাজে রাখে হয়।

Avg data rate to browsers = $\frac{\text{Web object size}}{\Delta} * \text{Avg Req rate}$

$\Delta = 1.5 \text{ mbps}$

Access link utilization = $\frac{\Delta}{\text{Access link rate}} = 0.97$

$\rho \approx 1 \rightarrow \text{queuing delay } (q)$

$\rho < 0.6 \rightarrow \text{mili.s}$

LAN utilization = $\frac{\Delta}{\text{LAN speed}} = 0.0015 \rightarrow (\mu s)$

* Utilization রকম = অর্থাৎ রকম মাত্রা।

end-to-end Delay = internet delay(RTT) + access link delay + LAN delay
 = 2s + min + $\mu \text{ sec}$

Soln:

⇒ Option 1: Buy faster access link

↙ costly

⇒ Option 2: Install a web cache.

↙ cache hit = 40% = 0.4

cache miss = 60% = 0.6

Rate to browser over access link = $0.6 \times 1.5 = 0.9$

$$\text{Access link utilization} = \frac{0.9}{1.54} = 0.58 \text{ (m sec)}$$

Avg end-to-end delay = $0.6 * (\text{delay from original server} + \text{queuing}) + 0.4 * \text{delay when cache hit}$

$$= 0.6 (2s + 0.01s) + 0.4 (ms)$$

milli s 3 लेखा 2000

$$= \sim 1.2s$$

↙ easier + less costly

Sadi Sir

Lecture-6

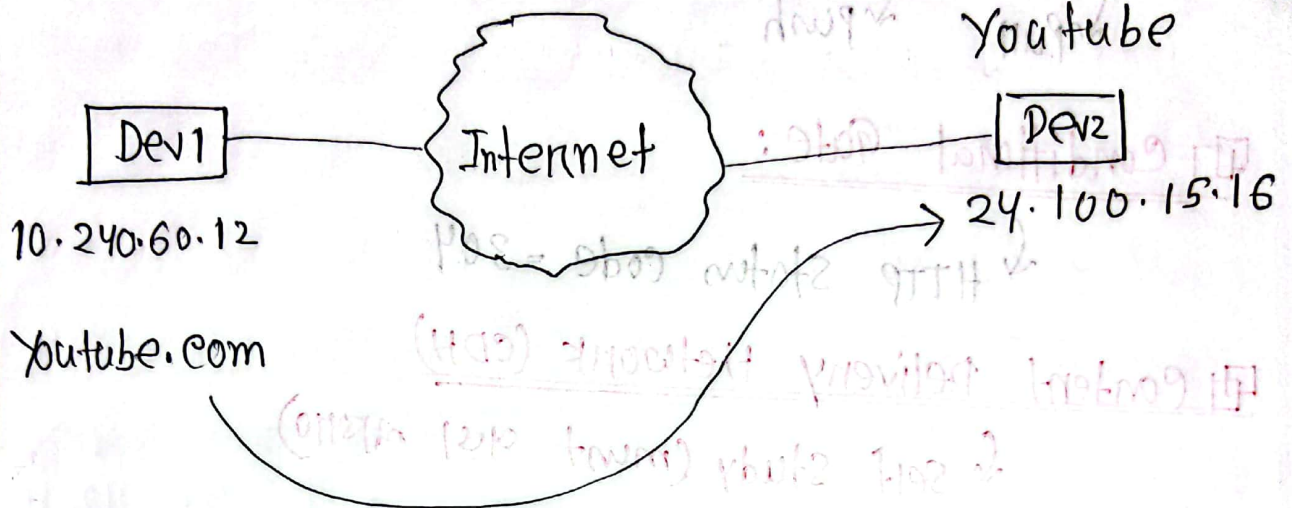
TOPIC NAME : DNS

DAY: Sunday

TIME:

DATE: 31, 8

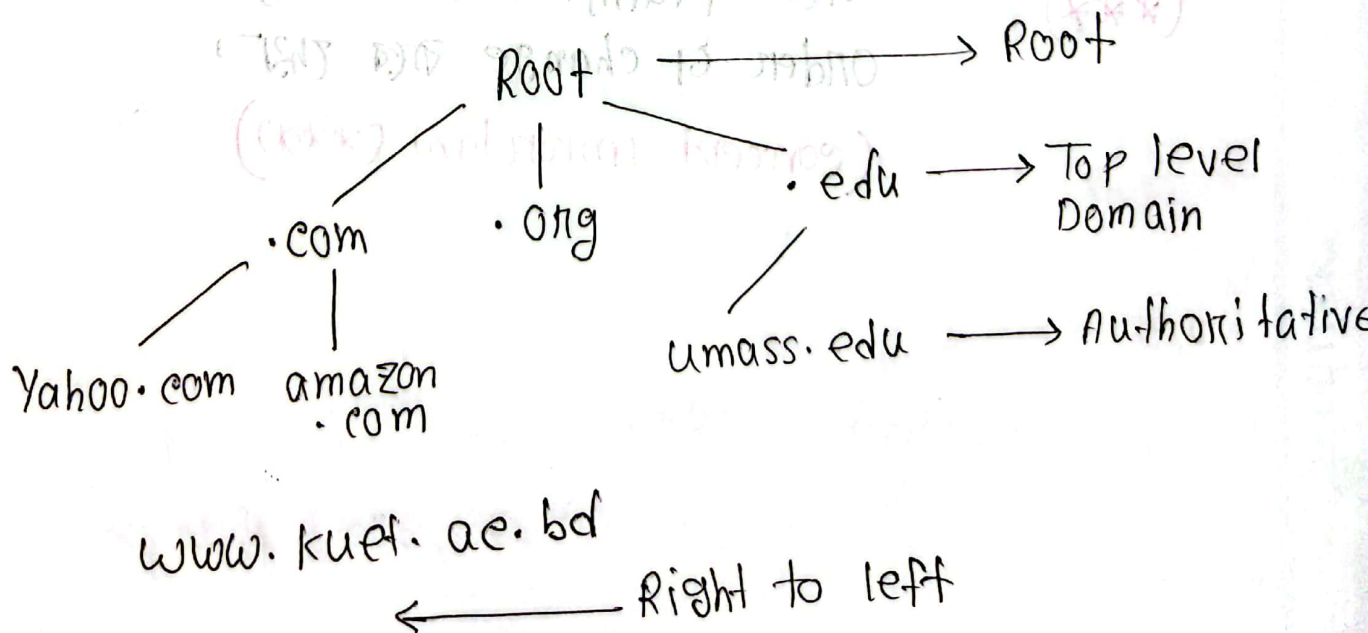
(*) Application layer ka kaam CT 1



(*) IP Address ka kaam Abstraction (Number) → String & map karna (DNS).

(*) DNS kaam application layer service.

(*) DNS - centralized or decentralized?



TOPIC NAME: _____

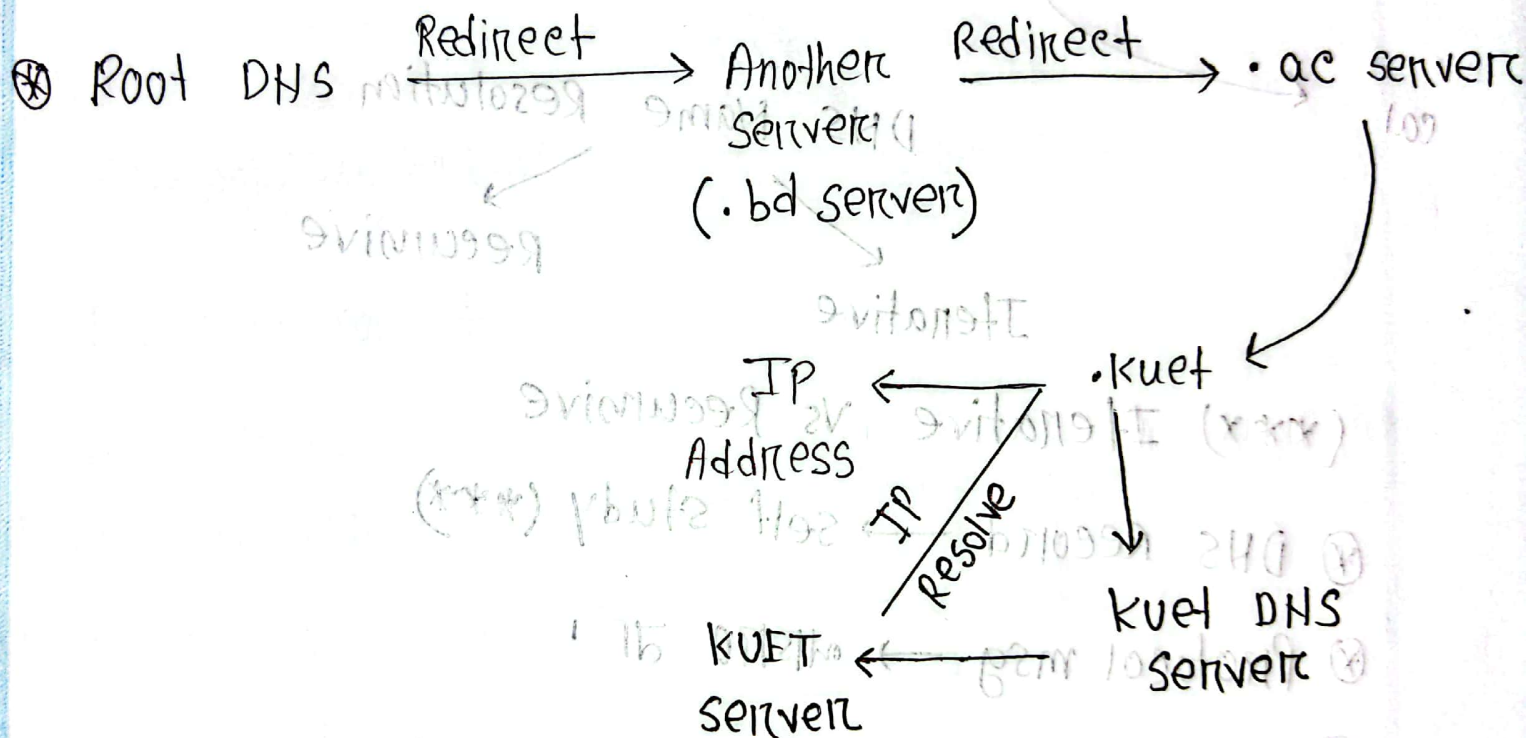
DAY: _____

TIME: _____

DATE: / /

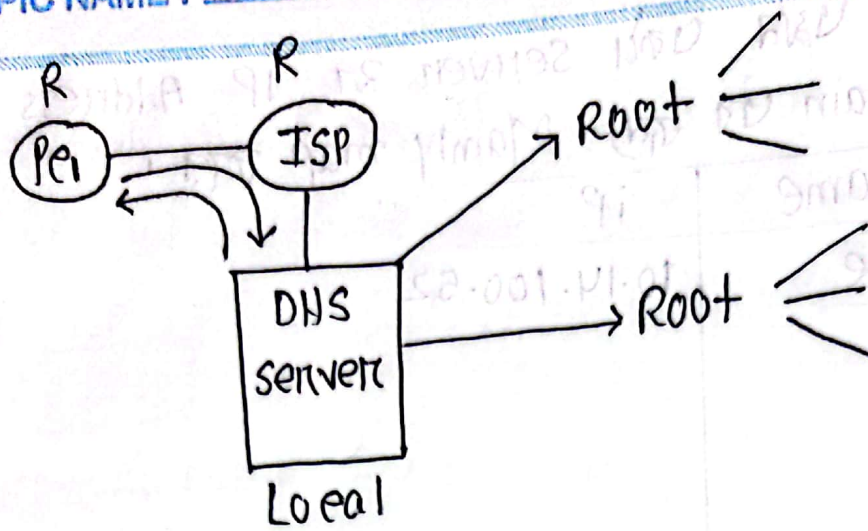
=> DNS server ଏକ ଏକ server ର IP Address provide କରେ, ଏକ Domain ଏକ କ୍ଷେତ୍ର। Mainly map କରେ।

name	ip
Youtube	10.14.100.52



⊗ Explain why DNS is Decentralized (***)

↓ ଏକ DNS server ଏକ ଅନେକ କଲପି ଅଟେ। ତାହା କାହିଁକି।
 ଏକ DNS server down ହେଲେ, ଅନ୍ୟ server ଏ hit କରା ଥାଏ।



DNS Name Resolution
 Iterative
 Recursive

(***) Iterative vs Recursive

- ⊗ DNS Record → self study (***)
- ⊗ Protocol msg → msg at
- ⊗

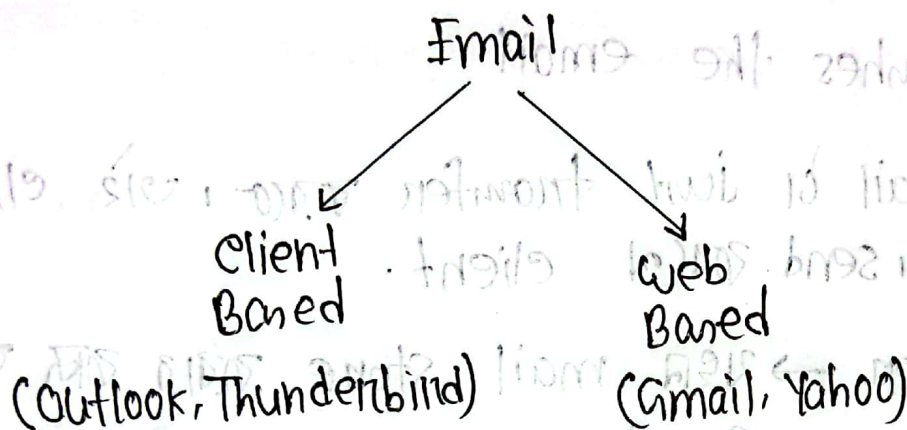
⊗ User Agent

⊗ MTA

⊗ MAA

↳ Outlook

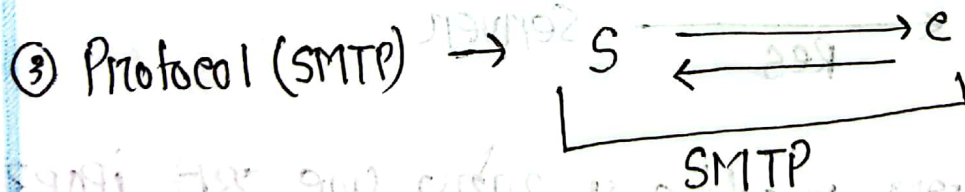
↳ Email client



✚ Email component

① User agent

② Mail server → Email. स्थिति store याम



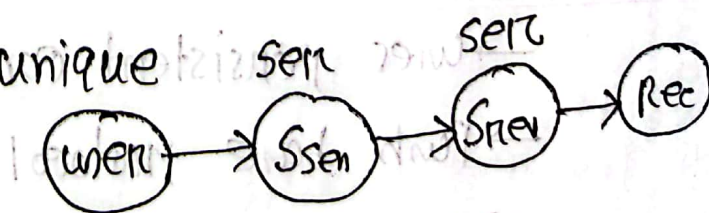
✚ Mail server:

① Mail Box → अलग रूप unique

② Mail queue →

③ SMTP

↳ Architecture.

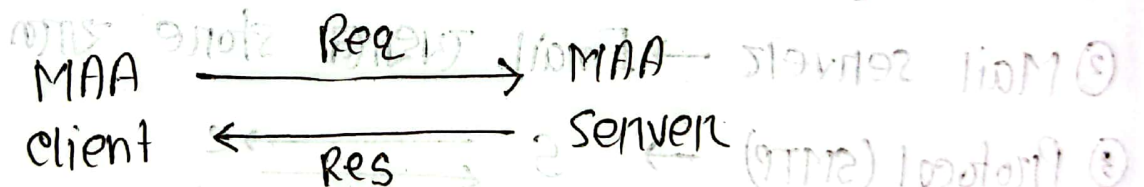


Email Architecture :

Senden, Receive Same mail server

- ⊗ SMTP → Senden 727n Rec 727 Server 7 72772727
→ Just pushes the email
- ⊗ MTA → Email 727 just transfer 727727, 727727 client 727727
727727 727727, send 727727 client.
- ⊗ MTA server → 727727 mail store 727727 727727 727727.
Receive 727727 server.

⊗ MAA → Message Access Agent



727 server

⊗ Receiver 727727 727 machine 7 727727 we 727727 POP3, IMAP.

⊗ SMTP :

- user persistent connⁿ
- Push base protocol
- Can send 7bit ASCII code
- SMTP + MIME

POP: Post Office Protocol

- Rx ও server তখন mail নিয়ে, client কে পাঠানো।
- Then mail server থেকে delete করে দিও।
- Multiple device থেকে access করা যায় না।

IMAP:

- Server থেকে copy করে client এ পাঠানো।
- Then client কে পাঠানো।
- Multiple device থেকে access করা যায়।
- Storage Problem

Web Based Email: (Self Study) → HTTP কীভাবে correlate করতে হবে।

→ Prev + HTTP transaction

→ HTTP server

→ যেহেতু HTTP দিয়ে শুধু, jml নাম টি Replace করে দিচ্ছি।

→ HTTP Abstraction, underline SMTP কে কাজ করে।

⊗ MIME just convert করা, যাও SMTP কে কাজে আসে।
 we করা হয়।

Sadi Sir

TOPIC NAME: Transport layer

DAY: Sunday
TIME: _____ DATE: 14

Introduction:

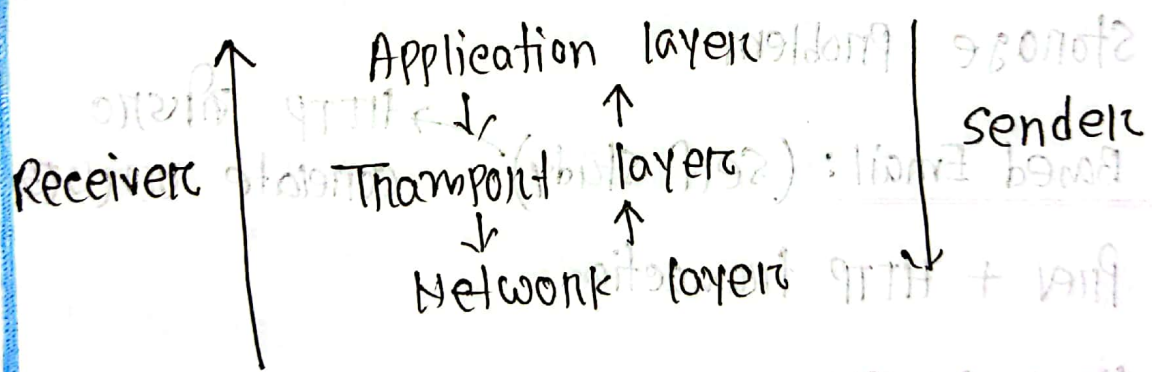
→ Obj: End device to end device msg transport করে।

Role of ~:

- Logical end to end transport.
- Process to process msg transport করে।

Protocol:

- ① TCP → Reliable
- ② UDP → Minimal packet loss accepted.



- ⊗ Pe to Pe ক্ষেত্রে → Network layer.
- ⊗ Pe কোম্পানি, but কত process → Transport layer.

Process to Process Delivery:

- Node to Node → Data link layer
- Host to host → Network layer
- Process to process → Transport layer

Router $\rightarrow$ L3

Switch/hub $\rightarrow$ L1 $\rightarrow$ Switch mainly mac address
wire $\rightarrow$ L1 check করে।

Addressing :

① Data link $\rightarrow$ MAC Address

② Network $\rightarrow$ IP

③ Transport $\rightarrow$ Port number (16 bit)

(*** HTTP, port = 80, বাকী কী দুমার?

Connⁿ Control:

① Connⁿ oriented $\rightarrow$ TCP

② Connⁿ less $\rightarrow$ UDP

Service:

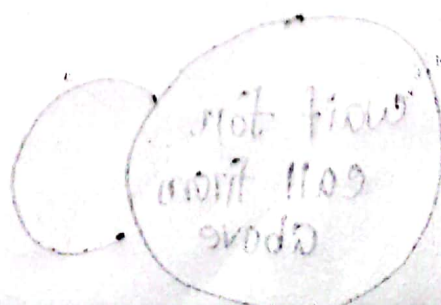
① Reliable

$\rightarrow$ Error control

$\rightarrow$ Flow control

$\rightarrow$ Congestion "

Tcp implement করে, UDP implement করে না।



Sadi Sir

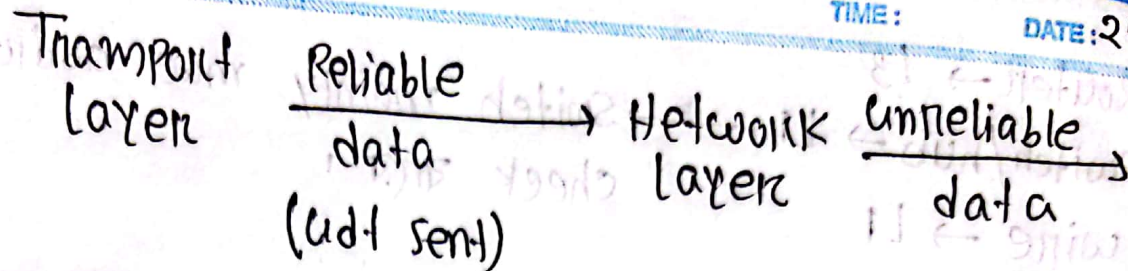
Lecture

TOPIC NAME: Reliable Data

DAY: Sunday

TIME:

DATE: 21/



- * Rdt send
- * Udt send
- * rdt-receive
- * deliver data

* We will use finite state Machine (FSM)

$\Rightarrow \frac{A}{B} \rightarrow$ $\frac{A}{B}$ is state variable (cause)
 $\rightarrow$ Action, state change karu or action perform karu shes.

* Send() or udt-send()

Model 1: RDT 1.0

Assumption -

- ① No error
 - ② No packet loss
- } Ideal channel

③

App layer

Sender

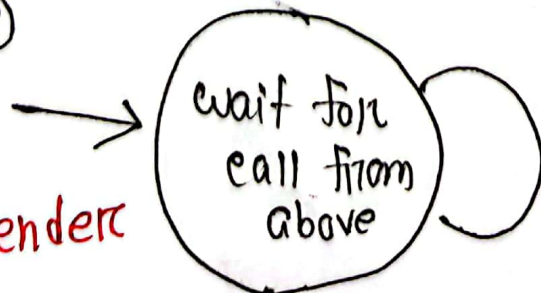


Diagram showing the receiver side:

came $\rightarrow$ data $\rightarrow$ rdt-receive $\rightarrow$ packet = make-pkt(data) $\rightarrow$ udt-send(packet) $\rightarrow$ Action

TOPIC NAME: _____

DAY: _____

TIME: _____

DATE: / /

Receiver

wait for call

Mode 2: rdt 2.0 -

Assumption -

① channel with bit error.

New concept introduced -

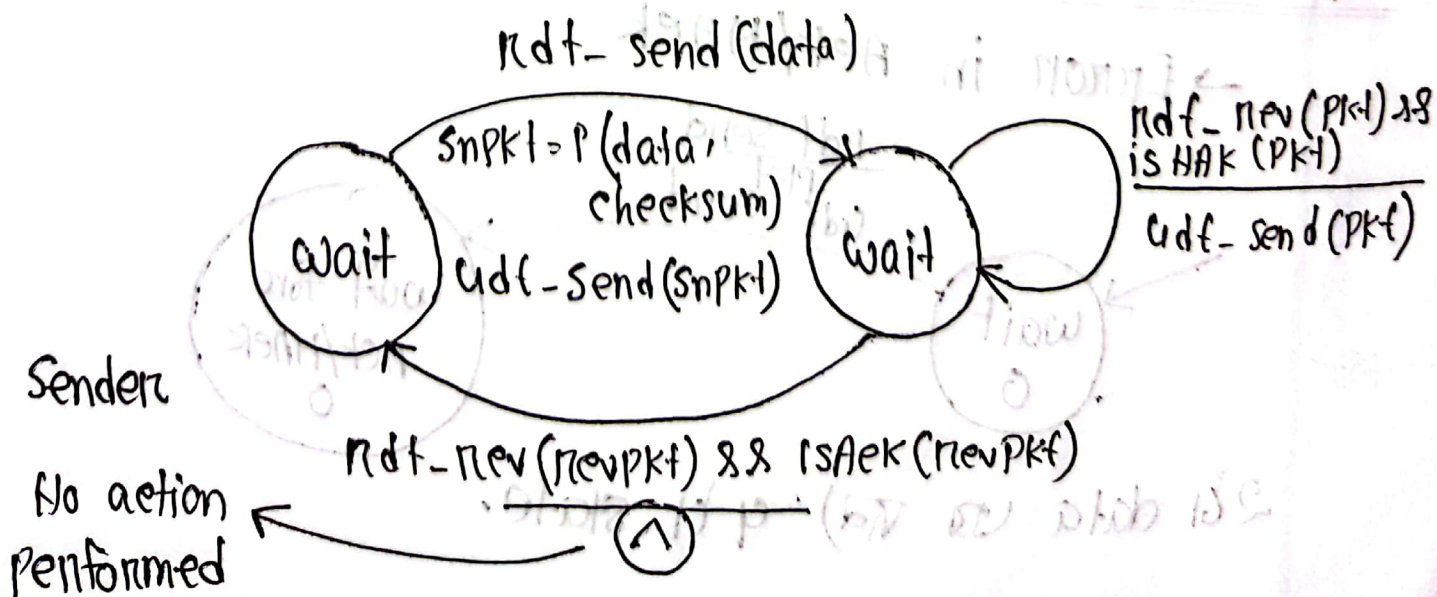
① Ack ② Negative Ack

↪ ફોન રજો NAK

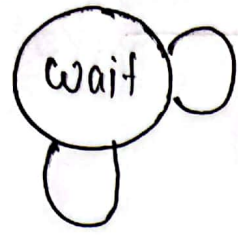
Receiver error check કરાવો, checksum ત્રિધર ।

ફોન તારે Ack send કરાવો ।

→ Message Pass/ Commⁿ રજો Rx & Tx વચ્ચે સરિયાર ।



Receiver



$\text{ndf} - \text{rev}(\text{newpkt})$ s's corrupt (newpkt)
 $\text{udf} - \text{send}(\text{HAK})$

$\text{ndf} - \text{rev}(\text{PKT})$ s's notconn(PKT)
 $\text{ext}(\text{PKT}, \text{data})$
 $\text{deliver data}(\text{data})$
 $\text{udf} - \text{send}(\text{ACK})$

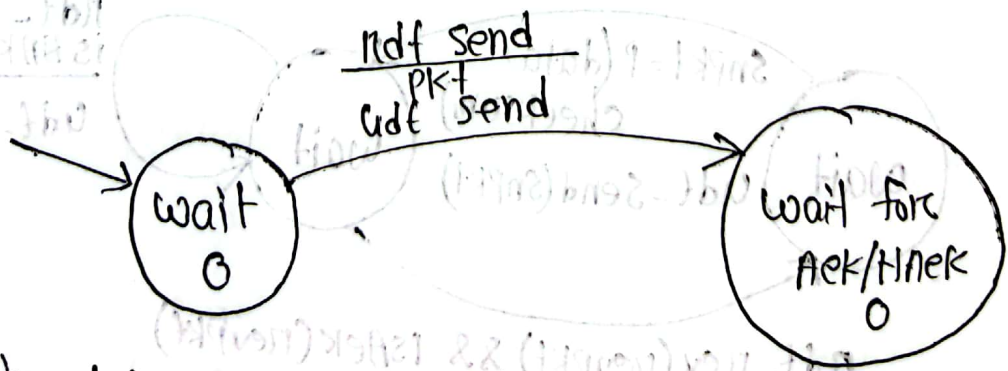
❓ ACK/HACK 2 vīn corrupted 2(25 25)?

Soln ↓ Packet v seq num add krā hō.

⊗ Stop and wait Protocol.

❓ Ndf 2.1:

→ Error in ACK/HACK



2 bī data vā krā y bī state.

TOPIC NAME :

DAY :

TIME :

DATE : / /

Ndf 2.1 : Discussion -

① Why 0.1 is used ? (***)

Model 4: Ndf 2.2:

→ Same as 2.1

→ NAck તો, Positive Ack નીચે કાલ કરે ।

→ isAck + seq No use કરે ।

TCP Segment Structure: (***)

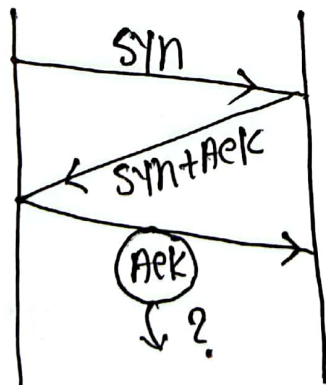
segment structure আনক কিছু explain করা আসাও। এর type math তেও আসাও exercise তাক। (***)

⊗ Ack 78 আসে 0-78 clear, ওহা 79 packet আকার করে।

⊗ seq 92 = আমি ওহে 92 নং packet send করছি।

⊗ Ack = seq + ৩০ Byte Data

⊗ TCP ২ way handshake ব্যবহার করলে কী সমস্যা হতো, কিন্তু 3 way handshake করা হয়। কিন্তু extra Ack আসে ?
(***) কিন্তু 4-way আসে না ?



(***) 2 way handshake scenarios → চিহ্ন তাক

(***) Half open connection.

(***) 3-way handshake

File Numbering System :

Data = 5k Bytes

segment = 5

Per segment = 1k Bytes

Any Random number

Seg 1 → seg No : 10,001 Range : 10,001 - 11,000

Seg 2 → seg No : 11,001 Range : 11,001 - 12,000

seg no + no of bytes

Header (20-60 bytes)



⇒ 8001 আসে → 8000। এর আরা 30 Aek করা হয়েছে।

⇒ 3rd stage এ আবার 8000 কেন? why not 8002?

↓ Same packet এ Aek + Data send করা হয়।
Concept Piggybacking

→ মনে রাখা Aek sequence এর কোন Data না থাকে
আর অন্য প্যাকেট sequence বহন করে, করা হয়।

(***) 3-way handshake [Sir question into mind]

⊗ Conn Terminate করার F flag use করা হয়।

⊗ 3 way communication এর মাধ্যমে Conn terminate
করা হয়।

Ques:

① 3 way HS

② Seq no কখন বাড়ে, কখন বাড়ে না।

③ কীভাবে terminate করা হয়।

④

Top Flow control

- ⊗ Receive Window Flag
- ⊗ Receiver, Sender to control
- ⊗ Dynamic sliding window
- ⊗ 3rd Commⁿ Bi-directional
- ⊗ Sender + Receiver $\rightarrow$ Send + Recv Buffer
- ⊗ What is send Buffer?

Send window:Receive window:

* Receive Buffer (or) shrink 200 kb ?

⊗ Spare room in buffer

$$= \text{adRev W} \bullet$$

$$= \text{Rev Buf} - [\text{LastByteRe} - \text{LastRead}]$$

$$= 100 - (260 - 200) = 40$$

$$\otimes (\text{LastByteSent} - \text{LastByteRecd}) \leq \text{RecvWindow}$$

⊗ (****) An example of flow control [Page-21]

⊗ Page-23 An example exam 4

⊗ Prevent the shrinking of the send window -

$$\text{new ackNo} + \text{new rwnd} \geq \text{last ackNo} + \text{last rwnd}$$

⇒ কোন কমাতে পারে না।

⊗ Silly window syndrome:

Sender { window size অনেক ছোট।
soln: Nagle's algo

Receiver { Received 1 Byte করে consume করে।
soln: Flood algo

⇒ Delayed Acknowledgement

TOPIC NAME: Error Control

Q ~ Tool:

- ① checksum
- ② Ack
- ③ Timeout

Q Rules for Generating Ack:

(*** Page-10 এর Diagram

- ① Packet এর সাথে Ack send, expect ack till then.
- ② Packet এ Ack send করার পরে 500ms wait করে।
- ③ Max^m দুইটা Packet পর্যন্ত wait করা।
- ④ Packet in order না হওয়া পর্যন্ত application layer এ data send করতে না।

scenario নিয়ে নিচের question }
এ

Q Fast Retransmission:

→ Jina Packet এর জনে data ও duplicate Ack আসলে sender wait করে না, direct এ Packet send করে দিবে।

Q Lost Ack:

Higher number Ack পাচ্ছে, যদিও ok, সাথে Ack এ matter না।

Congestion Control

⇒ H/W ও অনেক traffic, অনেক delay, এখানে solve করা
 দরকার।

এই Top Woes:

① ~ window ② ~ policy

Tyes:

① End to end ~ ② H/W assistant

↓ Top এখানে implement করে।

⊗ $\text{LastByteSent} - \text{LastByteAck} \leq \min\{\text{CongWin}, \text{RecvWin}\}$

↓ Data send করা always min^m in

এই sender কীভাবে বুঝে H/W ও congestion আছে?

⇒ Assume করা থাকে। Actually, আছে না, Timeout

নির্দেশ assume করে।

↓ Packet lost

↓ Packet ওহেনও যায় না। NID গিয়েও পায় না।

⊗ Error control ব্যবস্থায়, same কারণে congestion

বিস্তার হয়।

↓
 End to End

TOPIC NAME : _____

DAY: _____

TIME: _____

DATE: _____

જા H/w assistant હ એ ડુબિરી સુમા આદે । Router જ
જાનાય તય H/w અતેક load રહેય આદે । H/w
assist કાય ।

જા Size of CongWindow:

- ① slow start
- ② Additive increase
- ③ Multiplicative decrease

જા TCP slow start:

1MSS = 500 bytes

જા Additive increase:

Max^m window size + 1

(***) Page - 33 explain with all condition.

(*) TCP cubic તાહી.

(***) TCP fair તિ તા ?

TCP - RTT Estimation.

* Timeout $>$ RTT $\rightarrow$ Condition 1

4 Types of timer:

① Retransmission timer

② Persistence Times

③ Keepalive Times

④ Time wait times

* TCP 70 এর timer

শাক।

Oldest Ack এর জন্য timer

শাক।

Retransmission Timer:

Main focus of topic is

Jacobson's Algorithm for estimate timeout:

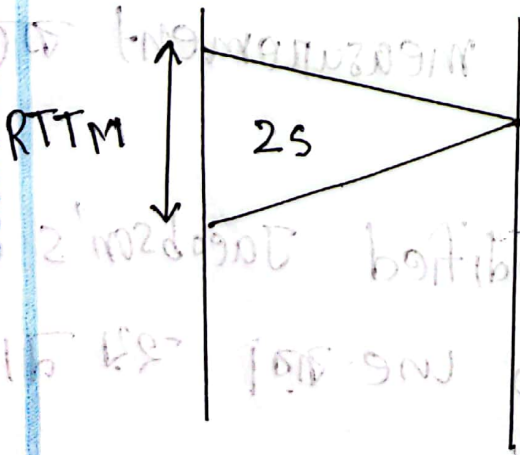
$RTT_M = \text{Measured time}$ RTT

$RTT_S = \text{Estimated RTT}$ $\rightarrow$ প্রায় RTT নিয়ে কাজ করা। RTT Avg/something

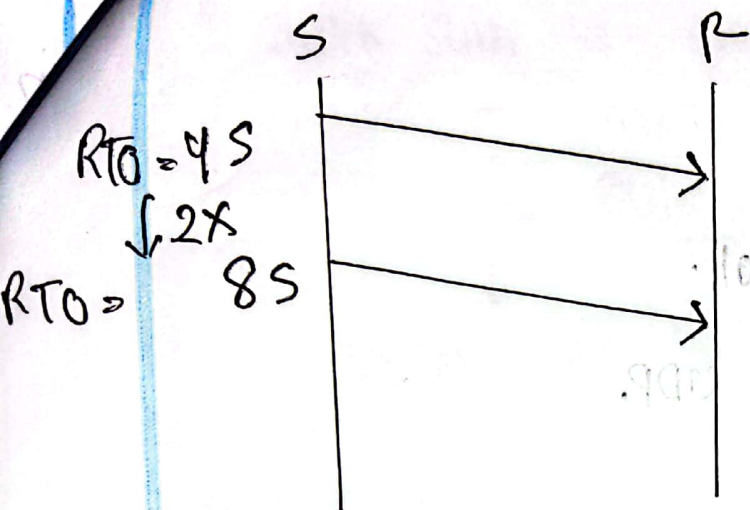
কিছু করে।

$$RTT_S = (1-\alpha) RTT_S + \alpha \times RTT_M$$

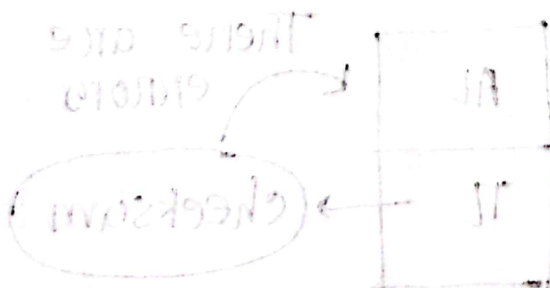
$$\alpha = 1/8 = 0.125$$



Kahn's Algorithm:



* Jacob पर उलगा की? कारण me क्या है ता।
Kahn's को solve करे। Kahn कारण शायद।



Sadi Sir

Lecture -

TOPIC NAME: UDP & QUIC

DAY: 26/10/25

TIME:

DATE: Sun

⊗ UDP is connectionless.

⊞ Why is there a UDP:

① No connection

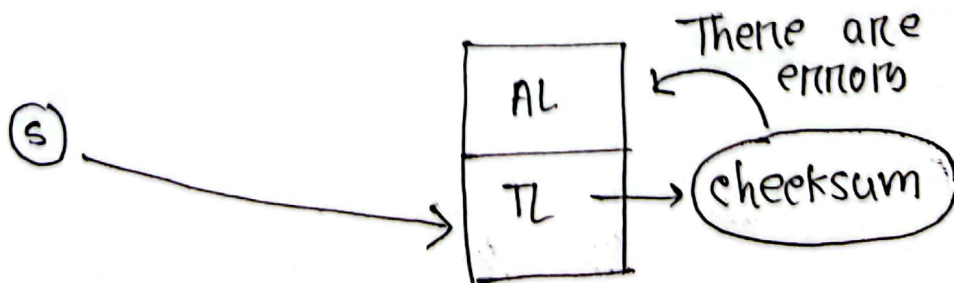
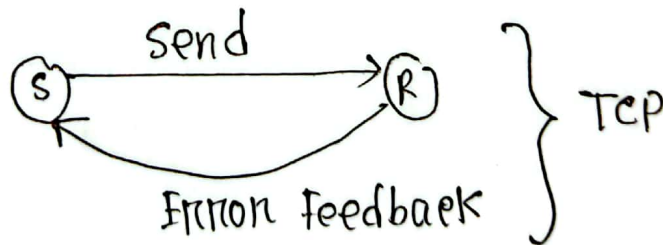
② Simple

③ No congestion, no flow control.

→ DNS, SNMP, HTTP/3 uses UDP.

⊞ UDP segment header:

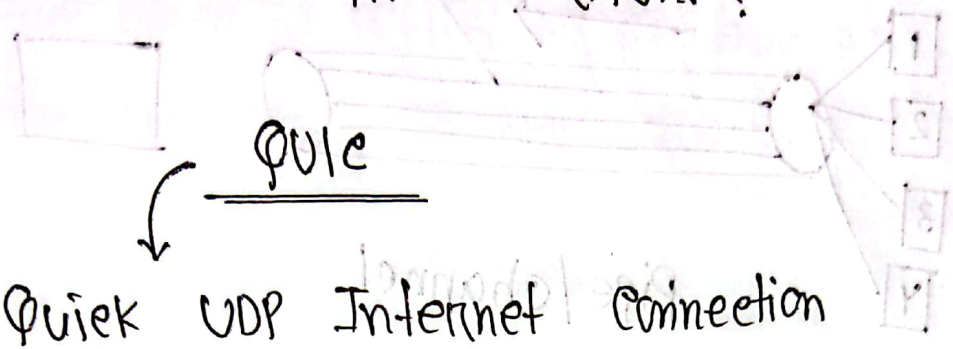
⊞ UDP check the error in different way.



⊗ Then AL then decide, what will done with error bit. May be it may discard it.

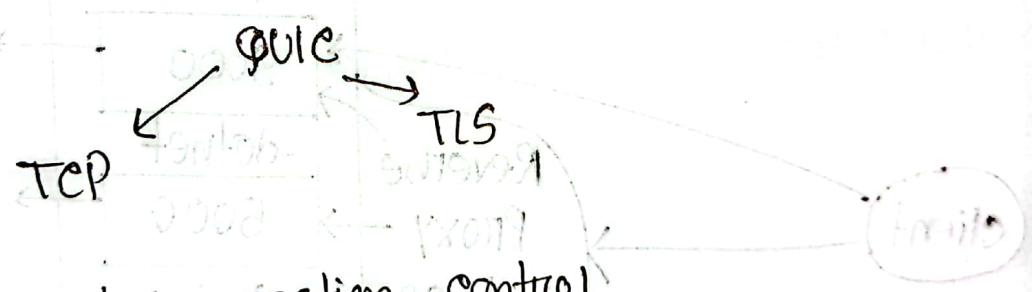
Weeksum calculation 'অবসর' তা exam.

⊗ Check sum & Problem সমস্যা?



⊗ TLS (Transport layer security)

↳ Data change করে হয়, কিন্তু দেখে বুঝা যায় না। Internal Structure change হয়।



① Error and congestion control

② Connection establishment

⊗ TCP ৩৩ Head of line Blocking আছে।

1	2	3	4	5	6	7
---	---	---	---	---	---	---

② X

২ আর ৩ তে, ওই Data শাও তা Head layer এ।

localhost = 127.0.0.1

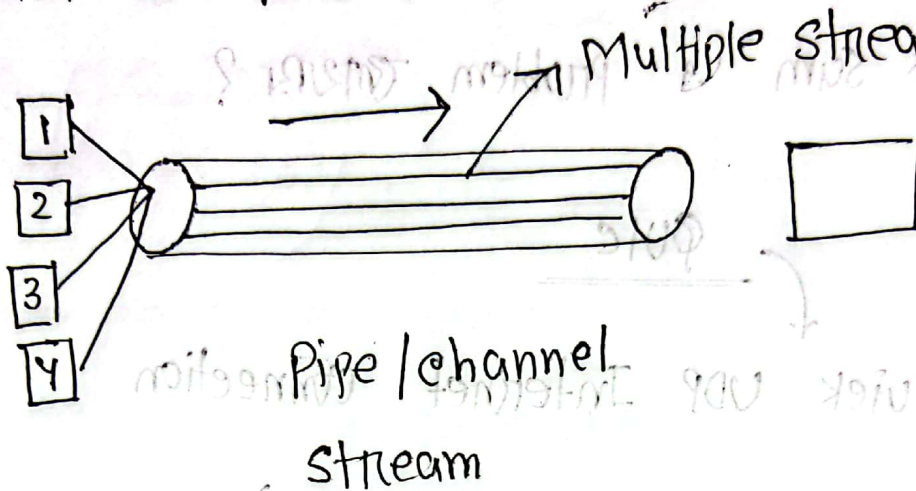
TOPIC NAME : _____

DAY : _____

TIME : _____

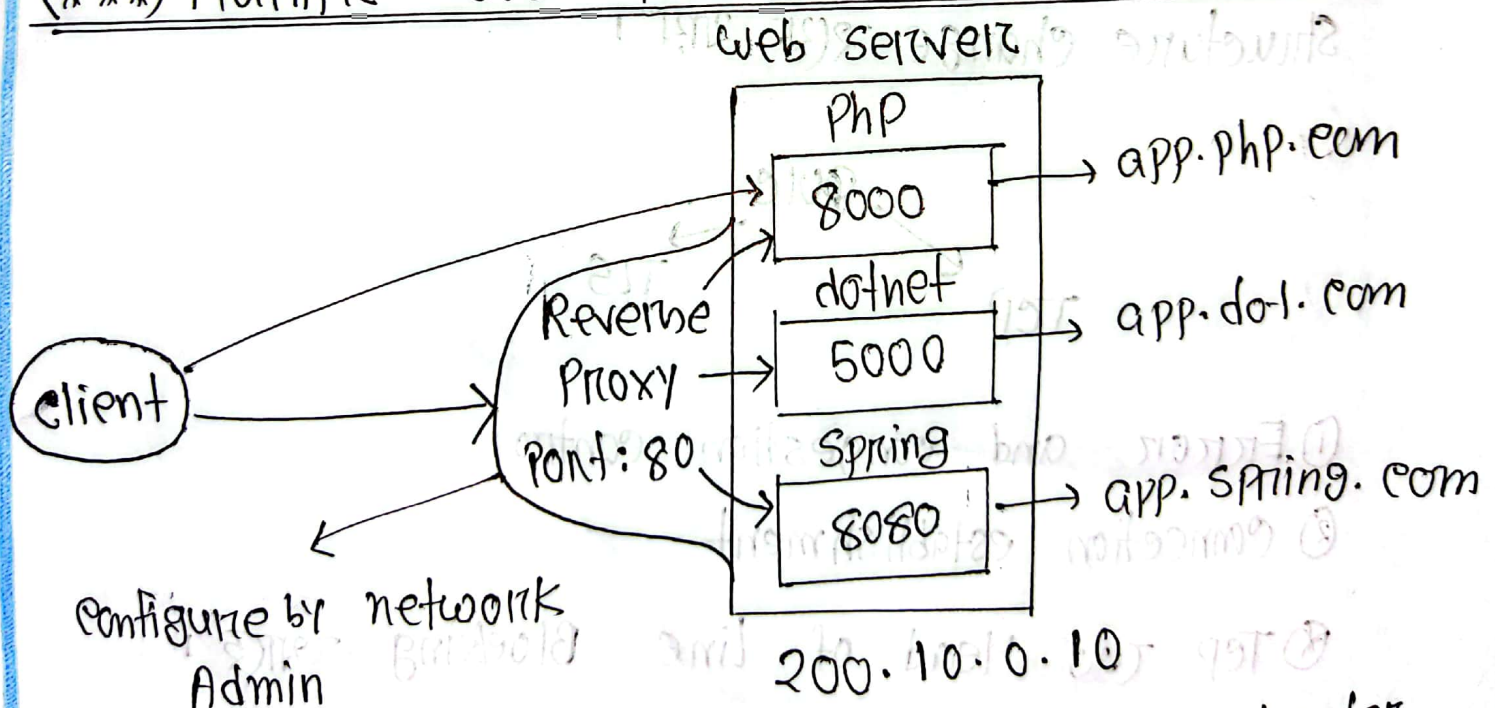
DATE : _____

*** What is pipe? What type of Protocol uses?



→ Multiple stream এর জন্য dependency নাই

(*** Multiple web apps in a single server :



Some.com : 80 → HTTP

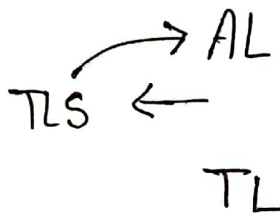
Some.com : 443 → HTTPS

⊗ Reverse proxy if header
থেকে Port exact করে mapping
করে।

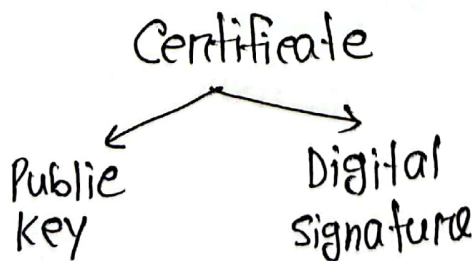
⊗ Domain name mapping করে
করে।

Question :

- ① Per set 3-4 questions, question অনেক বেশি থাকবে।
- ② Scenario Based, যা পড়ানো হয়েছে topic ওয়ে।
- ③ Directly বলা থাকবে না topic, বুঝে নিতে হবে।

Transport layer security

- ⊗ Encrypted way to Data বোঝানো।
- ⊗ At first establish TCP connection.
- ⊗ যখনই TCP connⁿ established হবে, তখন encrypted data send করতে।

TLS 1.2 Handshake: (***)

- ⊗ First এ we use public key
- ⊗ Last এ we use symmetric key.

- ⊗ Symmetric key → fast
- ⊗ Asymmetric key → slow
- RSA

⊗ SSL (***)